The Journal of Open Source Software

aprende-vslam: a level-based Visual SLAM course in Spanish where every claim is a verified number

Ariel Eliezer Vazquez Dominguez [1]

1 Independent Researcher, Mexico

DOI: N/A

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: 01 January 1970

Summary

aprende-vslam is a practical Visual SLAM course in Spanish, organized as 25 self-contained levels that take a student from *"an image is a matrix of numbers"* to a complete visual SLAM system with bundle adjustment and loop closure, and beyond: real RGB-D and stereo datasets (Burri et al., 2016; Sturm et al., 2012), learned features (DeTone et al., 2018; Lindenberger et al., 2023), real-time engineering, a differentiable 3D Gaussian Splatting rasterizer built from scratch (Kerbl et al., 2023), a ROS 2 integration, and four bonus levels on estimation theory — factor graphs, IMU preintegration (Forster et al., 2017), Kalman filtering from first principles, and the machinery inside GTSAM (Dellaert & Kaess, 2017; Kaess et al., 2012): variable elimination, elimination ordering, the Bayes tree, and a working toy iSAM.

The course's central design decision is that **every claim ends in a number, and every number has an automatic exam**. Each level ships a `verificacion.py` script — 202 checks across the 25 levels — that compares the student's implementation against measured expected values: a hand-written grayscale conversion must match OpenCV within ± 1 intensity level; marginalization must equal the Schur complement to 4×10^{-16}; the full TUM `fr2_xyz` sequence must track at 1.4 cm ATE with zero lost frames; removing bundle adjustment must collapse accuracy by more than an order of magnitude (5.8 cm $\rightarrow$ 148 cm in the canonical run). Passing the exam is the definition of mastering the level.

Statement of need

Visual SLAM sits at the intersection of geometry, optimization, probability and software engineering, and the canonical learning resources present either the theory or a finished system. Textbooks such as Hartley & Zisserman (2004) give the mathematics; video lecture series such as Stachniss (2020) give the intuition; reference systems such as ORB-SLAM2 (Mur-Artal & Tardós, 2017) give a state-of-the-art implementation to run. The resource closest in spirit, Gao & Zhang (2021), ships companion code for each chapter — but progress there, as in course repositories generally, is verified by inspection: the student compiles, runs, and judges that *"the trajectory looks right"*. That leaves two gaps. First, the student can read the math and can run the system, but rarely gets to *build the bridge* between them and check each plank under their own weight. Second, verification by inspection does not scale to self-study, where no instructor is present to say whether a result is actually correct.

aprende-vslam addresses both gaps at once:

1. **Verification as assessment.** The 202 automated checks turn qualitative milestones into quantitative, reproducible ones, making the course usable without an instructor: the exam either passes or names the number that failed. Classic failures are part of the curriculum, reproduced on purpose and measured — the wrap-around bug in a heading innovation (39× worse), a one-sided axis-convention conversion (120° of constant false

attitude), the half-pixel convention bug in a differentiable rasterizer (29 dB).

2. **The Spanish-language gap.** There is very little in-depth, hands-on SLAM material in Spanish; essentially all of the resources above assume English. The course is written natively in Spanish (code, comments, exams and readmes), while remaining runnable and reviewable by non-Spanish speakers: the exams print numbers.

The intended audience is advanced undergraduates, graduate students and self-taught engineers with basic Python and introductory linear algebra; computer vision is explicitly *not* a prerequisite, since the course builds it from the sensor up.

Learning objectives

On completing the core blocks (A–C, levels 00–15), the student can:

- implement the image-formation pipeline from scratch — sensor noise model, pinhole projection, rigid-body poses, lens distortion and camera calibration — and validate each piece numerically against OpenCV;
- estimate relative camera motion from two views (feature detection, matching with the ratio test, epipolar geometry with RANSAC) and quantify its error against ground truth;
- build a complete visual SLAM system — triangulation, PnP over a persistent map, bundle adjustment via the Schur complement, Sim(3) loop closure — and measure what each component buys, by ablation;
- run that system on real RGB-D benchmarks (Sturm et al., 2012) and evaluate it with the standard metrics (ATE, scale error), including recovering metric scale from depth;
- diagnose the classic failure modes — drift, scale ambiguity, angle wrap-around, axis conventions, the map-mirage — by reproducing and measuring each one.

The electives and the bonus arc (block D) add: stereo on EuRoC (Burri et al., 2016), learned features and their limits (DeTone et al., 2018; Lindenberger et al., 2023), profiling and real-time engineering, a differentiable 3DGS rasterizer (Kerbl et al., 2023), a verified ROS 2 bridge, and the estimation trilogy — batch smoothing, filtering, and incremental smoothing implemented on identical measurements (Dellaert & Kaess, 2017; Forster et al., 2017; Kaess et al., 2012).

Course content

The 25 levels are grouped in four blocks (Figure 1); each level closes with the verified milestone shown in Table 1.

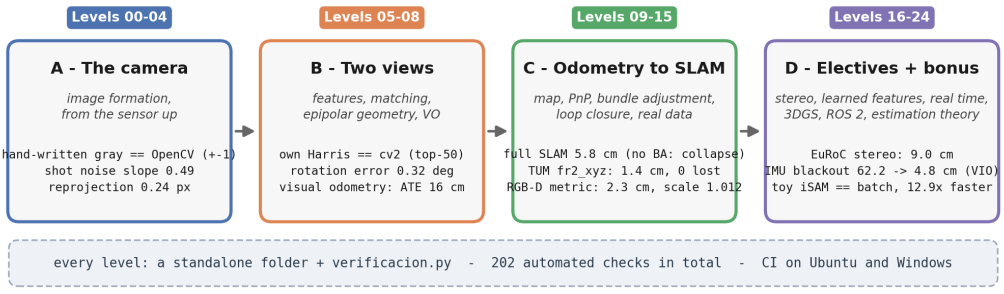

Figure 1: The arc of the course: four blocks, and the invariant that defines it — every level ends in a standalone exam.

Table 1: The 25 levels and the number each one closes with.

Block	#	Level	Verified milestone
A	00	Environment and first pixels	hand-written grayscale == OpenCV (± 1 level)
A	01	The image sensor	shot noise $\propto \sqrt{\text{signal}}$ (measured slope 0.49)
A	02	Pinhole camera	wireframe cube rendered in pure NumPy
A	03	Poses and transformations	the full round trip composes to the identity
A	04	Distortion and calibration	0.24 px reprojection; a bent line straightens 24×
B	05	Features	own Harris matches `cv2` on 100% of the top-50
B	06	Matching	ratio test: 30% → 8% incoherent matches
B	07	Epipolar geometry	0.32° rotation error against ground truth
B	08	Visual odometry	ATE 16 cm — and drift, watched growing
C	09	Triangulation	first 3D map, exported to `.ply`
C	10	PnP and a persistent map	18.6 → 8.7 cm changing only the architecture
C	11	Bundle adjustment	Schur complement + the 7-dof gauge, measured
C	12	Pose graph and loop closure	a monocular loop in SE(3) *worsens*; Sim(3) fixes it
C	13	Full SLAM	5.8 cm; without bundle adjustment, collapse (>10×)
C	14	Real data (TUM)	full `fr2_xyz`: 1.4 cm ATE, 0 lost frames
C	15	RGB-D and metric scale	`fr1_desk`: 2.3 cm *metric*, scale 1.012
D	16	Stereo (EuRoC)	drone `V1_01`: 9.0 cm, scale 1.004
D	17	Learned features	SuperPoint+LightGlue survives 13× more blur — and still loses `fr1_desk`
D	18	Real-time engineering	BA 4.4× faster at machine precision (4×10^{-16})
D	19	Dense mapping (3DGS)	own differentiable rasterizer: 58 dB; the half-pixel bug costs 29 dB
D	20	ROS 2	the bridge verified *without* ROS (9×10^{-16}); the one-sided axis bug: 120°
D	21	Factor graphs (bonus)	same measurements, five backends: full 8.2 < EKF 9.4 < window 25.9 < poses 29 < truncated 59 cm
D	22	The IMU factor (bonus)	visual blackout in a curve: 62.2 → 4.8 cm with preintegration (VIO)
D	23	The EKF from scratch (bonus)	KF == linear factor graph (10^{-11}); the $\pm\pi$ bug: 39× worse; ESKF at 7.2 cm online
D	24	GTSAM from scratch (bonus)	elimination == batch (7×10^{-15}); toy iSAM 12.9× faster; real GTSAM matches to 0.4 mm

Every level follows the same template: flat, top-to-bottom-readable scripts (one per "act"), a driver that prints the level's tables and figures, a readme with the intuition and the measured results, a set of exercises (120 across the course) each with its own numeric target, and the exam. Levels have zero imports between them and no shared package: any level can be copied alone to another machine and run.

Instructional design

Six rules act as the course's constitution; the three that define its character map directly onto instructional-design principles:

- **Mastery gating.** A level is "passed" when its exam passes — an operational form of mastery learning (Bloom, 1968), workable without an instructor precisely because the criterion is a number. The exercises extend this with retrieval practice (Roediger &

Karpicke, 2006): each of the 120 exercises states a target number the student must reproduce or beat, so practice is always testing.

- **Worked examples, then modification.** Each level is a worked example (Sweller & Cooper, 1985) decomposed from a working parent system: the student reads a flat script in which the mathematics lives beside the line that uses it (`── La matemática ──` blocks), runs it, and then modifies it under the exercises. Studying a correct worked implementation before problem solving is the pedagogical bet of the whole course.
- **Cognitive load over DRY.** Zero imports between levels; shared infrastructure is *deliberately duplicated* and trimmed to what the student knows at that point, keeping extraneous load low (Sweller, 1994) — immediate context beats software-engineering orthodoxy in teaching code.

A fourth rule governs honesty: when reality intrudes, the course documents and measures it instead of hiding it. GTSAM publishes no Windows wheel for current Python, so the final level runs the real library in a container and validates that its answer matches the from-scratch implementation to 0.4 mm of ATE.

What an exam looks like, verbatim (level 13, full SLAM):

```
Verificando sobre el corredor (200 frames)

  [OK ] el sistema inicializa y trackea sin perderse  (0 frames perdidos)
  [OK ] se insertaron >=10 keyframes  (18 (medido: 18))
  [OK ] el mapa tiene >=2000 puntos  (5246 pts (medido: 5246))
  [OK ] se cerro al menos 1 bucle  (2 bucles: [(4, 12), (1, 17)])
  [OK ] los bucles son LEJANOS en el tiempo (no con el vecino)

  ATE online:        8.8 cm
  ATE keyframes:     5.8 cm  <- la metrica honesta

  [OK ] ATE de keyframes < 15 cm  (5.8 cm (medido: 5.8))
  [OK ] sin BA el sistema COLAPSA (>10x peor)  (129.9 cm vs 5.8 cm con BA)
  [OK ] el cierre de bucle no empeora la trayectoria de keyframes
  [OK ] todo el mapa esta delante de la camara inicial  (100.0%)

NIVEL 13: VERIFICADO
```

Checks encode two kinds of claims: exact reproductions with explicit tolerances (the Schur identity to 4×10^{-16}), and *invariants* where the honest statement is a ratio or a threshold — the ablation above must collapse by more than $10\times$, whatever the exact figure of a given run. This distinction is what lets the exams re-verify on other machines and in CI without pretending more determinism than the algorithms have.

Experience of use and adoption

The course is designed natively for **unsupervised self-study**: the exam plays the role the instructor's judgment plays in a classroom, with the advantage that failure names the failing number. It has been developed and verified end-to-end by its author — every expected value in the 202 checks is a measurement, not an aspiration, and a maintainer script (`verifica_todos.py`) plus continuous integration re-run the exams on machines the author does not control. It has not yet been piloted with a formal student cohort; the design goal was precisely that adoption should not depend on the author being present.

For instructors, the independence rule makes adoption granular:

- **A single level as a lab assignment.** Any level is a standalone folder; `verificacion.py` doubles as an automatic grader, and the level's exercises with numeric targets are

ready-made extension work.

- **Blocks A–C as a semester core.** Sixteen levels take a cohort from pixels to a measured SLAM system on real data; block D provides project-sized electives.
- **The exams as a rubric.** Because milestones are numbers, grading criteria need not be invented: the course already states what "working" means, level by level.

Story of the project

The course is the pedagogical decomposition of a working parent system — `vslam-edu` on PyPI (Vazquez Dominguez, 2026) — built first, by the same author. *That repository is the destination; this one is the path.* The decomposition is why the milestones have provenance: many are the parent system's real measurements, including its failures. The sequence that defeated feature-based RGB tracking (`fr1_desk`) motivates the RGB-D level; the 13× blur robustness of learned features — and why it still was not enough — motivates the sensor lesson; the collapse without bundle adjustment is the parent system's own ablation, reproduced by every student who takes level 13. When a level says *"the parent repo measured this"*, it is literal, and it usually comes with the story of what it cost to discover.

Quality control

Continuous integration runs the dataset-free exams on Ubuntu and Windows on every push (two exams whose expected numbers are calibrated against OpenCV's Windows backend run on Windows CI only — an OpenCV platform-sensitivity the course documents rather than hides); the dataset levels are verified locally against TUM and EuRoC sequences, and `verifica_todos.py` runs all 25 exams end to end. The GTSAM container smoke test is exercised with an optional exam flag (`--docker`).

Acknowledgements

The course distills lessons from building its parent system, and openly builds on the ecosystems it teaches: OpenCV, NumPy, Matplotlib, PyTorch, GTSAM and ROS 2.

References

Bloom, B. S. (1968). Learning for mastery. *Evaluation Comment*, *1*(2), 1–12.

Burri, M., Nikolic, J., Gohl, P., Schneider, T., Rehder, J., Omari, S., Achtelik, M. W., & Siegwart, R. (2016). The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research*, *35*(10), 1157–1163. https://doi.org/10.1177/0278364915620033

Dellaert, F., & Kaess, M. (2017). Factor graphs for robot perception. *Foundations and Trends in Robotics*, *6*(1–2), 1–139. https://doi.org/10.1561/2300000043

DeTone, D., Malisiewicz, T., & Rabinovich, A. (2018). SuperPoint: Self-supervised interest point detection and description. *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. https://doi.org/10.1109/CVPRW.2018.00060

Forster, C., Carlone, L., Dellaert, F., & Scaramuzza, D. (2017). On-manifold preintegration for real-time visual-inertial odometry. *IEEE Transactions on Robotics*, *33*(1), 1–21. https://doi.org/10.1109/TRO.2016.2597321

Gao, X., & Zhang, T. (2021). *Introduction to visual SLAM: From theory to practice*. Springer. https://doi.org/10.1007/978-981-16-4939-4

Hartley, R., & Zisserman, A. (2004). *Multiple view geometry in computer vision* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511811685

Kaess, M., Johannsson, H., Roberts, R., Ila, V., Leonard, J. J., & Dellaert, F. (2012). iSAM2: Incremental smoothing and mapping using the Bayes tree. *The International Journal of Robotics Research*, *31*(2), 216–235. https://doi.org/10.1177/0278364911430419

Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, *42*(4). https://doi.org/10.1145/3592433

Lindenberger, P., Sarlin, P.-E., & Pollefeys, M. (2023). LightGlue: Local feature matching at light speed. *IEEE/CVF International Conference on Computer Vision (ICCV)*. https://doi.org/10.1109/ICCV51070.2023.01616

Mur-Artal, R., & Tardós, J. D. (2017). ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras. *IEEE Transactions on Robotics*, *33*(5), 1255–1262. https://doi.org/10.1109/TRO.2017.2705103

Roediger, H. L., & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, *17*(3), 249–255. https://doi.org/10.1111/j.1467-9280.2006.01693.x

Stachniss, C. (2020). *Online lectures on SLAM, photogrammetry and mobile robotics*. University of Bonn. https://www.youtube.com/@CyrillStachniss

Sturm, J., Engelhard, N., Endres, F., Burgard, W., & Cremers, D. (2012). A benchmark for the evaluation of RGB-D SLAM systems. *Proc. Of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. https://doi.org/10.1109/IROS.2012.6385773

Sweller, J. (1994). Cognitive load theory, learning difficulty, and instructional design. *Learning and Instruction*, *4*(4), 295–312. https://doi.org/10.1016/0959-4752(94)90003-5

Sweller, J., & Cooper, G. A. (1985). The use of worked examples as a substitute for problem solving in learning algebra. *Cognition and Instruction*, *2*(1), 59–89. https://doi.org/10.1207/s1532690xci0201_3

Vazquez Dominguez, A. E. (2026). *Vslam-edu: An educational visual SLAM system in Python*. PyPI / GitHub. https://github.com/ariel9874/Visual-slam